

Practical 4 (References)



Paint

- You can set the color of the **Paint** using **setColor()**
- Standard colors are defined within **android.graphics.Color**

- The colors can be set for the **Paint** objects

```
Paint redPaint = new Paint();  
redPaint.setColor(Color.RED);
```

- Once a color is set, all drawing methods called below will behave to adopt the same color unless another is set

Paint Styles

- Paint styles control how an object is filled with color
- Possibly values are FILL, STROKE or both

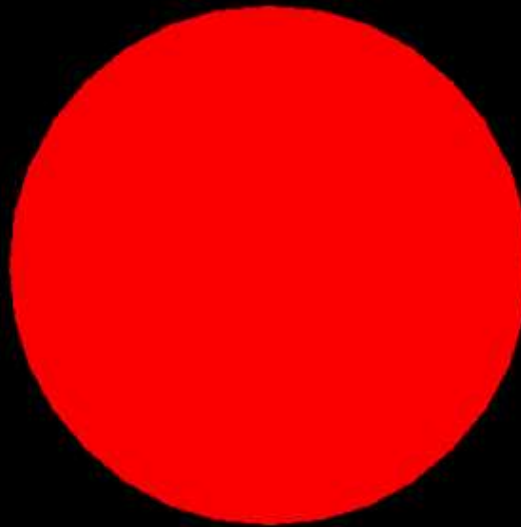
```
Paint linePaint = new Paint();  
linePaint.setStyle(Paint.Style.STROKE);
```

- Similar to color, every upcoming drawing method will draw according to the style set until another style was set

Simple Example

```
private static class ViewCustom extends View{  
    public ViewCustom(Context context){  
        super(context);  
    }  
  
    @Override  
    protected void onDraw(Canvas canvas) {  
        canvas.drawColor(Color.BLACK);  
        Paint circlePaint = new Paint();  
        circlePaint.setColor(Color.RED);  
        canvas.drawCircle(canvas.getWidth()/2, canvas.getHeight()/2,  
            canvas.getWidth()/3, circlePaint);  
    }  
}
```

ViewExample





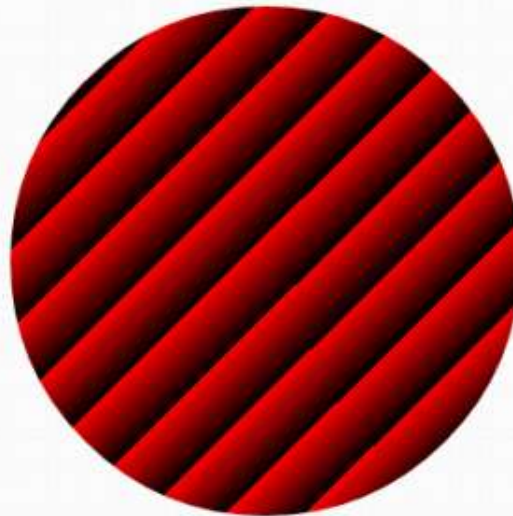
Paint Gradient

- Colors does not need to always be solid, they can have gradient
- E.g. **LinearGradient**, **RadialGradient** and **SweepGradient**
- These are under **android.graphics.Shader**
- A gradient is defined by at least two colors, a starting color and an ending color with probably more color in within
- The **setShader()** method is used on **Paint** to set the gradient

Gradient - example

```
@Override
protected void onDraw(Canvas canvas) {
    Paint circlePaint = new Paint(Paint.ANTI_ALIAS_FLAG);
    LinearGradient linGrad = new
        LinearGradient(0,0,25,25,Color.RED,Color.BLACK,
            Shader.TileMode.REPEAT);
    circlePaint.setShader(linGrad);
    canvas.drawCircle(canvas.getWidth()/2,
        canvas.getHeight()/2,canvas.getWidth()/3,circlePaint);
}
}
```

ViewExample





Drawing Text

- We can use the drawing capability to draw text instead of showing them using TextView
- The flexibility in controlling the styles and where to show the text is stronger
- The user can choose the font, style, size and locations while painting certain text

Typeface

- First you have to define a Typeface with selected Typeface family and font style
- You can the set the Typeface and font size to the **Paint** object that you had created

```
Typeface type =  
    Typeface.create(Typeface.SANS_SERIF,  
    Typeface.NORMAL) ;  
mPaint.setTypeface (type) ;  
mPaint.setTextSize (16) ;
```

Typeface- example

```
Typeface ftypes[] = {Typeface.MONOSPACE,Typeface.SANS_SERIF,Typeface.SERIF};
int atypes[] =
    {Typeface.NORMAL,Typeface.BOLD,Typeface.ITALIC,Typeface.BOLD|Typeface.ITALIC};
String ftypes_str[] = {"Monospace","Sans Serif","Serif"};
String atypes_str[] = {" (Normal)"," (Bold)"," (Italics)"," (Bold & Italics)"};

Paint mPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
Typeface mType;

mPaint.setTextSize(16);

for(int i=0; i<atypes.length; i++){
    for(int j=0; j<ftypes.length; j++){
        mType = Typeface.create(ftypes[j],atypes[i]);
        mPaint.setTypeface(mType);
        canvas.drawText(ftypes_str[j] + atypes_str[i], 20, 20*(i+1)+(j*80),
            mPaint);
    }
}
```

ViewExample

Monospace (Normal)
Monospace (Bold)
Monospace (Italics)
Monospace (Bold & Italics)
Sans Serif (Normal)
Sans Serif (Bold)
Sans Serif (Italics)
Sans Serif (Bold & Italics)
Serif (Normal)
Serif (Bold)
Serif (Italics)
Serif (Bold & Italics)

Other styles

- Some properties of a typeface are set using **setFlags()** method instead of defining a Typeface
- E.g. antialiasing, underlining and strike-through
- These styles could be used together with the styles defined in the Typeface, such as Normal, Bold and Italics
- `mPaint.setFlags(Paint.UNDERLINE_TEXT_FLAG) ;`

Other styles - example

```
Typeface ftypes[] = {Typeface.MONOSPACE, Typeface.SANS_SERIF, Typeface.SERIF};
int otypes[] =
    {Paint.ANTI_ALIAS_FLAG, Paint.UNDERLINE_TEXT_FLAG, Paint.STRIKE_THRU_TEXT_FLAG};
String ftypes_str[] = {"Monospace", "Sans Serif", "Serif"};
String otypes_str[] = {" (Antialiasing)", " (Underline)", " (Strike through)"};

Typeface mType;
mPaint.setTextSize(16);

for(int i=0; i<otypes.length; i++){
    for(int j=0; j<ftypes.length; j++){
        mType = Typeface.create(ftypes[j], Typeface.NORMAL);
        mPaint.setTypeface(mType);
        mPaint.setFlags(otypes[i]);
        canvas.drawText(ftypes_str[j] + otypes_str[i], 20, 20*(i+1)+(j*60),
            mPaint);
    }
}
```

ViewExample

Monospace (Antialiasing)
Monospace (Underline)
~~Monospace (Strike through)~~
Sans Serif (Antialiasing)
Sans Serif (Underline)
~~Sans Serif (Strike through)~~
Serif (Antialiasing)
Serif (Underline)
~~Serif (Strike through)~~

Dealing with Images

- Android allows images to be imported in the application
- Copy and place the image file under “**res/drawable**”
- Different resolutions are made available to optimize the resources for different screens (**ldpi/mdpi/hdpi/xhdpi**)
- The recommended file type would be **PNG (Portable Network Graphics)**

Drawing Images

- Canvas also allows the drawing of existing bitmap images using **drawBitmap()** method

- Example:

```
Bitmap pic = BitmapFactory.decodeResource(  
    getResources(), R.drawable.mountain);  
canvas.drawBitmap(pic, 10, 100, null);
```

- The images can be scaled using **createScaledBitmap()**

- Example:

```
Bitmap spic =  
    Bitmap.createScaledBitmap(pic, 300, 500, false);
```

ViewExample



ViewExample



Geometrical transformations

- **Matrix** class could be used to perform transformations on **Bitmap** graphic
- E.g. mirroring, rotation, translation and etc
- The **Matrix** helps to define the necessary transformation from the original image
- **createBitmap()** is used to apply the transformation on the image

Example

```
Matrix mirrorMatrix = new Matrix();  
mirrorMatrix.preScale(-1, 1);  
Matrix rotateMatrix = (-1new Matrix());  
rotateMatrix.preRotate(30);
```

```
Bitmap mirrorPic =  
    Bitmap.createBitmap(pic, 0, 0, pic.getWidth(), pic.getHeight(),  
        mirrorMatrix, false);  
Bitmap rotatePic =  
    Bitmap.createBitmap(pic, 0, 0, pic.getWidth(), pic.getHeight(),  
        rotateMatrix, false);
```

```
canvas.drawBitmap(mirrorPic, 10, 10, null);  
canvas.drawBitmap(pic, 10, 250, null);  
canvas.drawBitmap(rotatePic, 10, 450, null);
```

ViewExample



Displaying Shapes

- The drawn shapes can be regarded as Drawable which could be display as an image
- In High Level UI, Drawables could be displayed using an **ImageView**
- A similar set of primitives to draw such shapes in Canvas are also available
- Therefore shapes could be drawn for the Canvas or to be used in High Level UI as well

Rectangles

- Rectangles and squares can be drawn by specifying the height and width
- Squares are merely rectangles with equal width and height
- Drawn using **RectShape**
- The width/height of the rectangle is controlled by setting the intrinsic width/height

Rectangle - example

```
import android.graphics.drawable.ShapeDrawable;  
import android.graphics.drawable.shapes.RectShape;
```

```
ShapeDrawable rect = new ShapeDrawable(new RectShape());  
rect.setIntrinsicHeight(3);  
rect.setIntrinsicWidth(5);  
rect.getPaint().setColor(Color.CYAN);
```

```
ImageView iv = new ImageView(this);  
iv.setImageDrawable(rect);
```

```
LayoutParams params = new LayoutParams(300,300);  
iv.setLayoutParams(params);
```

ViewExample



Rounded Rectangles

- Rectangles with rounded corners
- Drawn using **RoundRectShape**
- The rounded corners could be represented using a float array of 8 values to represent the x,y pair for all 4 corners
- All 4 rounded corners can be designed to have different degree of arc on the corners
- Useful for designing custom buttons

Rounded Rectangle - example

```
import android.graphics.drawable.ShapeDrawable;  
import android.graphics.drawable.shapes.RoundRectShape;
```

```
ShapeDrawable roundRect = new ShapeDrawable(new  
    RoundRectShape(new float[] {5,5,5,5,5,5,5,5},  
        null, null));
```

```
roundRect.setIntrinsicHeight(50);
```

```
roundRect.setIntrinsicWidth(100);
```

```
roundRect.getPaint().setColor(Color.RED);
```

```
ImageView iv = new ImageView(this);
```

```
iv.setImageDrawable(roundRect);
```

ViewExample



Inner-Rounded Rectangles

- The Rounded Rectangles can be set to contain an inner rounded rectangle
- The inner radii is defined similar to the outer radii that is used in regular Rounded Rectangles
- An inset is used to defined the thickness of each edge to the inner rectangle
- The outcome will resembles the boundary of a Rounded Rectangle and the thickness of the boundary were according to the inset

Inner-Rounded - example

```
float[] outerRadii = new float[] {5,5,5,5,5,5,5,5};  
RectF insetRectangle = new RectF(10,10,10,10);  
float[] innerRadii = new float[] {5,5,5,5,5,5,5,5};  
  
ShapeDrawable roundRect = new ShapeDrawable(new  
    RoundRectShape(outerRadii,insetRectangle,innerRadii));  
roundRect.setIntrinsicHeight(50);  
roundRect.setIntrinsicWidth(100);  
roundRect.getPaint().setColor(Color.GREEN);  
  
ImageView iv = new ImageView(this);  
iv.setImageDrawable(roundRect);
```

ViewExample



Ovals

- The ovals and circle can be drawn using this idea
- Circles are merely oval with equal width and height
- Drawn using **OvalShape**
- The size of the oval is controlled by setting the intrinsic width and height
- The curve will be automatically generated within the defined area to form the oval

Oval - example

```
import android.graphics.drawable.ShapeDrawable;  
import android.graphics.drawable.shapes.OvalShape;
```

```
ShapeDrawable oval = new ShapeDrawable(new  
    OvalShape());  
oval.setIntrinsicHeight(50);  
oval.setIntrinsicWidth(100);  
oval.getPaint().setColor(Color.YELLOW);
```

```
ImageView iv = new ImageView(this);  
iv.setImageDrawable(oval);
```

ViewExample



Arcs

- Arcs are used to draw incomplete oval/circle shapes
- Drawn using **ArcShape**
- The arcs are defined by the starting angle and the degree of sweep for the arc shape
- 0 degree always starts from the 3 o'clock direction
- The sweep is following clockwise direction unless negative values are used

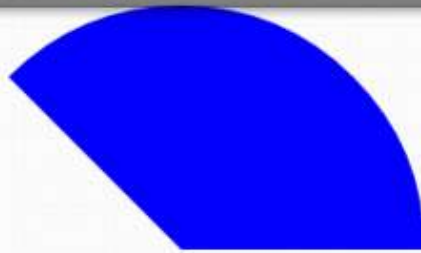
Arc - example

```
import android.graphics.drawable.ShapeDrawable;  
import android.graphics.drawable.shapes.ArcShape;
```

```
ShapeDrawable arc = new ShapeDrawable(new  
    ArcShape(0,-135));  
arc.setIntrinsicHeight(50);  
arc.setIntrinsicWidth(50);  
arc.getPaint().setColor(Color.BLUE);
```

```
ImageView iv = new ImageView(this);  
iv.setImageDrawable(arc);
```

ViewExample





Paths

- Paths can be used to draw other shapes that are more complicated based on series of lines and curves
- The path requires import of **android.graphics.Path**
- Paths drawn can be used to form a **PathShape**
- The PathShapes are defined by the designed Path and the width/height of the shape

Path - example

```
import android.graphics.Path;
```

```
Path p = new Path();  
p.moveTo(50, 0);  
p.lineTo(15, 100);  
p.lineTo(100, 35);  
p.lineTo(0, 35);  
p.lineTo(85, 100);  
p.lineTo(50, 0);
```

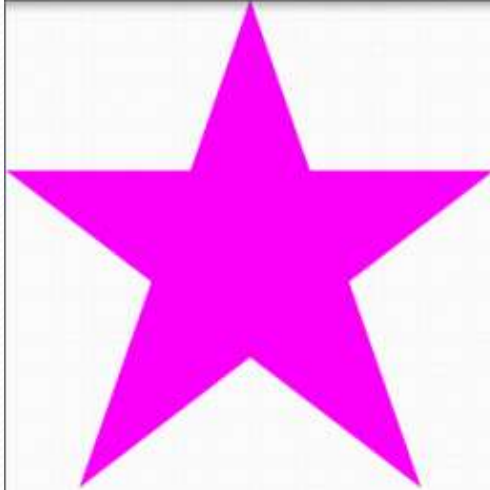
PathShape - example

```
import android.graphics.drawable.ShapeDrawable;  
import android.graphics.drawable.shapes.ArcShape;
```

```
ShapeDrawable star = new ShapeDrawable(new  
    PathShape(p, 100, 100));  
star.setIntrinsicHeight(100);  
star.setIntrinsicWidth(100);  
star.getPaint().setColor(Color.MAGENTA);
```

```
ImageView iv = new ImageView(this);  
iv.setImageDrawable(star);
```

ViewExample



Drawing Outlines

- Besides filling in the shapes, only a one pixel thick outline could be drawn for all the shapes by using style
- The style used is **Paint.Style.STROKE** for drawing outlines

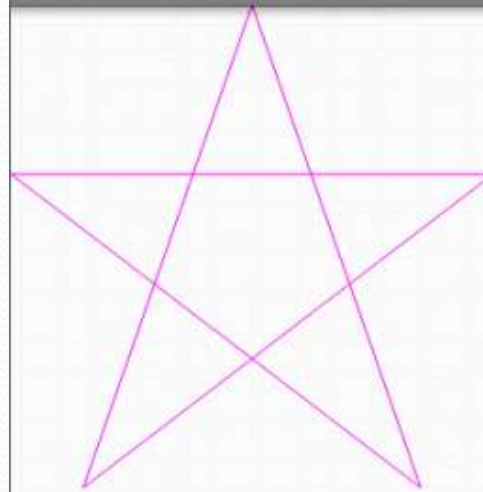
- Example:

```
shape.getPaint().setStyle(Paint.Style.STROKE);
```

ViewExample



ViewExample



Using XML

- Like many other aspect of Android, XML could be used for shape drawing as well
- Shape Drawables could be defined as XML under **/res/drawable/** directory
- The xml, like those used for layout, is used to describe the shape in terms of type and behavior (e.g. color)
- It can thereafter be imported as drawables to be displayed on ImageViews

XML - example (green_rect.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android=
    "http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#0f0" />
</shape>
```

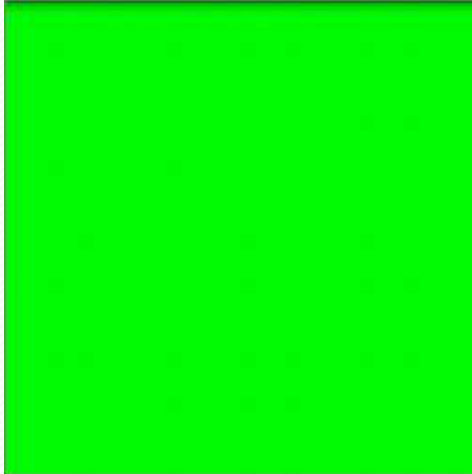
Displaying XML

```
ImageView iv = new ImageView(this);  
iv.setImageResource(R.drawable.green_rect);
```

```
LayoutParams params = new  
    LayoutParams(300, 300);  
iv.setLayoutParams(params);
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(iv);  
setContentView(ll);
```

ViewExample





Drawing Shapes in Canvas

- Shapes can also be drawn in the Canvas (Custom Views)
- Hint: Look for corresponding **drawXxx** methods that will help you accomplish it